

National Institute of Technology Karnataka Surathkal
Department of Information Technology



IT 252 DATABASE SYSTEMS

Concurrency Control Techniques

Dr. Jayashree T R

Course Outline

Course Plan: Theory:

Part A: Parallel Computer Architectures

Week 1: Introduction to the course, highlighting the data and databases, and related basic concepts. Advantages and need for a database system.

Week 2: Attributes, tuples, relational schema, conceptual model, introduction to SQL-DML, DDL, creating relations/ tables, access and manipulate data.

Week 3,4: Data model, relational model, and SQL, relational data model and relational database constraints, SQL data definition and data types, specifying constraints, basic retrieval queries, complex queries, triggers, views, schema modification.

Week 5-6: ER model development, entity types/sets, attribute, relationship types/sets, simple employee database conceptual design using ER concepts, ER to relational mapping algorithm, EER model.

Course Outline

- Week 7,8 : Functional dependency definition, need of normalization, anomalies and redundant information in tuples, normalization steps (1NF, 2NF, 3NF), BCNF, introduction to the higher normal forms.
- Week 9: Procedural query languages: relational algebra and SQL-SELECT, PROJECT, RENAME, binary and JOIN operations with examples.
Non-Procedural query languages: relational calculus, and SQL-tuple relational calculus, existential and universal quantifier with examples.
- Week 10,11 : Transaction management, schedule and serializability, concurrency control, 2-phase lock, recovery mechanism: undo/redo values.
- Week 12,13: Disk storage- basic file structures, ordered/unordered. binary search, hashing, indexing, importance of indexes, primary and secondary indexing methods, clustered, B tree, B++ trees.
- Week 14: Current trends in database system, introduction to data warehousing, data mining.
- Practical: MySQL commands
- Project: Team of 2 or 3 members

Introduction

- Concurrency control protocols
 - Set of rules to guarantee serializability
- Two-phase locking protocols
 - Lock data items to prevent concurrent access
- Timestamp
 - Unique identifier for each transaction
- Multiversion concurrency control protocols
 - Use multiple versions of a data item
- Validation or certification of a transaction

Two-Phase Locking Techniques for Concurrency Control

- A **lock** is a variable associated with a data item that describes the status of the item with respect to possible operations that can be applied to it.
- Generally, there is ***one lock for each data item*** in the data base.
- Locks are used as a means of synchronizing the access by concurrent transactions to the database items.

Types of Locks and System Lock Tables

- Several types of locks are used in concurrency control.
 - binary locks
 - shared/exclusive locks—also known as read/write locks

Binary locks

Binary locks

- Two states (values)
 - Locked (1)
 - Item cannot be accessed
 - Unlocked (0)
 - Item can be accessed when requested

Lock and unlock operations for binary locks

- Two operations, lock_item and unlock_item, are used with binary locking.
 - Transaction requests access by issuing a lock_item(X) operation

```
lock_item(X):  
B:  if LOCK(X) = 0          (*item is unlocked*)  
    then LOCK(X) ← 1      (*lock the item*)  
    else  
        begin  
            wait (until LOCK(X) = 0  
                  and the lock manager wakes up the transaction);  
            go to B  
        end;  
unlock_item(X):  
    LOCK(X) ← 0;          (* unlock the item *)  
    if any transactions are waiting  
        then wakeup one of the waiting transactions;
```

Figure 21.1 Lock and unlock operations for binary locks

Lock Tables

- Lock table specifies items that have locks
- Lock manager subsystem
 - Keeps track of and controls access to locks
 - Rules enforced by lock manager module
- At most one transaction can hold the lock on an item at a given time
- Binary locking too restrictive for database items

Shared/Exclusive (or Read/Write) locks

- **Multiple-mode lock**, also called shared/exclusive or read/write locks—there are three locking operations: `read_lock(X)`, `write_lock(X)`, and `unlock(X)`.
- A lock associated with an item X, `LOCK(X)`, now has three possible states: read-locked, write-locked, or unlocked.
- A **read-locked item** is also called **share-locked** because other transactions are allowed to read the item, whereas a **write-locked item** is called **exclusive-locked** because a single transaction exclusively holds the lock on the item.

Shared/Exclusive (or Read/Write) locks

Shared/exclusive or read/write locks

- Read operations on the same item are not conflicting
- Must have exclusive lock to write
- Three locking operations
 - `read_lock(X)`
 - `write_lock(X)`
 - `unlock(X)`

Figure 21.2 Locking and unlocking operations for two-mode (read/write, or shared/exclusive) locks

```
read_lock(X):
B: if LOCK(X) = "unlocked"
    then begin LOCK(X) ← "read-locked";
        no_of_reads(X) ← 1
    end
else if LOCK(X) = "read-locked"
    then no_of_reads(X) ← no_of_reads(X) + 1
else begin
    wait (until LOCK(X) = "unlocked"
        and the lock manager wakes up the transaction);
    go to B
end;

write_lock(X):
B: if LOCK(X) = "unlocked"
    then LOCK(X) ← "write-locked"
else begin
    wait (until LOCK(X) = "unlocked"
        and the lock manager wakes up the transaction);
    go to B
end;

unlock (X):
if LOCK(X) = "write-locked"
    then begin LOCK(X) ← "unlocked";
        wakeup one of the waiting transactions, if any
    end
else if LOCK(X) = "read-locked"
    then begin
        no_of_reads(X) ← no_of_reads(X) - 1;
        if no_of_reads(X) = 0
            then begin LOCK(X) = "unlocked";
                wakeup one of the waiting transactions, if any
            end
    end
end;
```

Guaranteeing Serializability by Two-Phase Locking

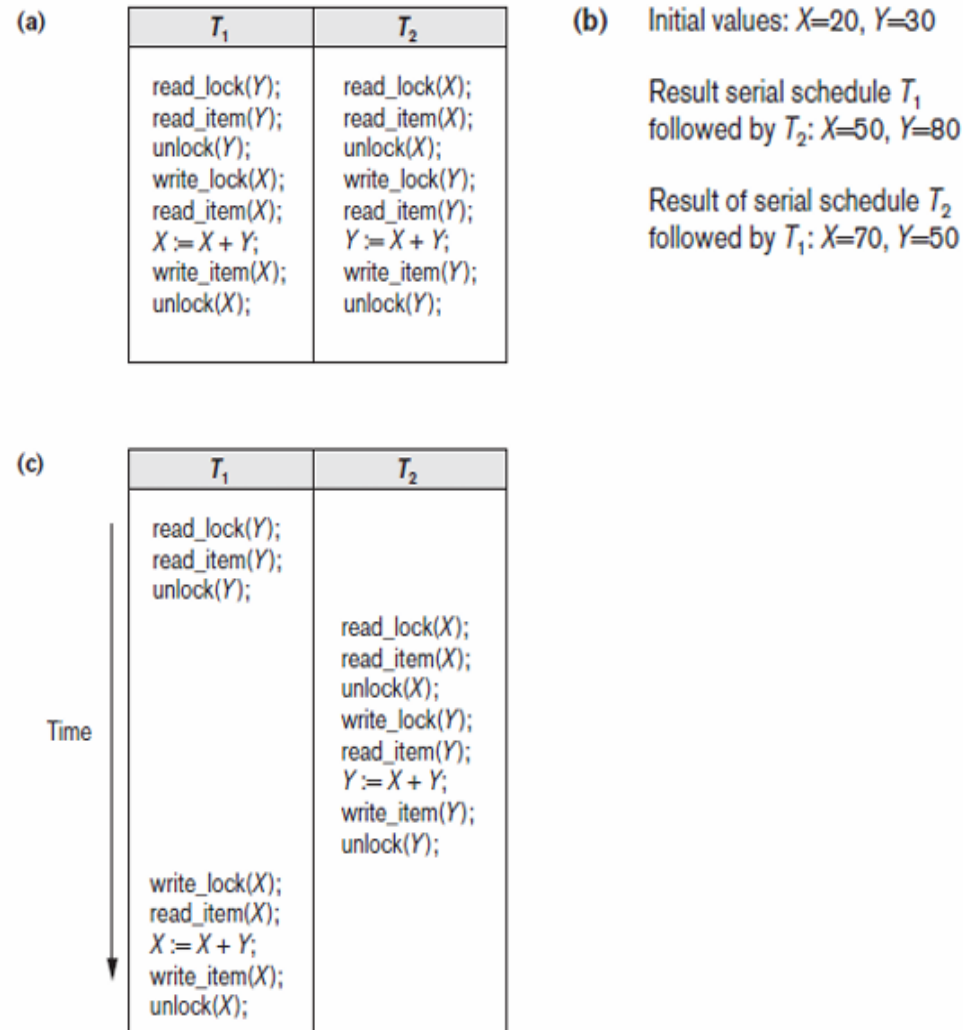
- A **transaction** is said to follow **the two-phase locking protocol** if all **locking operations** (**read_lock**, **write_lock**) precede the first **unlock operation** in the transaction.
- Such a transaction can be divided into **two phases**: an **expanding** or **growing (first) phase**, during which new locks on items can be acquired but none can be released;
- and a **shrinking (second) phase**, during which existing locks can be released but no new locks can be acquired.
- If lock conversion is allowed, then upgrading of locks (from read-locked to write-locked) must be done during the expanding phase, and downgrading of locks (from write-locked to read-locked) must be done in the shrinking phase.

Guaranteeing Serializability by Two-Phase Locking

- Two-phase locking protocol
 - All locking operations precede the first unlock operation in the transaction
 - Phases
 - Expanding (growing) phase
 - New locks can be acquired but none can be released
 - Lock conversion upgrades must be done during this phase
 - Shrinking phase
 - Existing locks can be released but none can be acquired
 - Downgrades must be done during this phase

Guaranteeing Serializability by Two-Phase Locking

Figure 21.3 Transactions that do not obey two-phase locking (a) Two transactions T_1 and T_2 (b) Results of possible serial schedules of T_1 and T_2 (c) A nonserializable schedule S that uses locks



Transactions T_1 and T_2 in Figure 21.3(a) do not follow the two-phase locking protocol because the write_lock(X) operation follows the unlock(Y) operation in T_1 , and similarly the write_lock(Y) operation follows the unlock(X) operation in T_2 .

Guaranteeing Serializability by Two-Phase Locking

Figure 21.4

Transactions T_1' and T_2' , which are the same as T_1 and T_2 in Figure 21.3 but follow the two-phase locking protocol. Note that they can produce a deadlock.

T_1'	T_2'
read_lock(Y);	read_lock(X);
read_item(Y);	read_item(X);
write_lock(X);	write_lock(Y);
unlock(Y)	unlock(X)
read_item(X);	read_item(Y);
$X := X + Y;$	$Y := X + Y;$
write_item(X);	write_item(Y);
unlock(X);	unlock(Y);

- If we enforce two-phase locking, the transactions can be rewritten as T_1' and T_2' , as shown in Figure 21.4.
- Now, the schedule shown in Figure 21.3(c) is not permitted for T_1' and T_2' (with their modified order of locking and unlocking operations) because T_1' will issue its `write_lock(X)` before it unlocks item Y; consequently, when T_2' issues its `read_lock(X)`, it is forced to wait until T_1' releases the lock by issuing an `unlock(X)` in the schedule, leading to a deadlock.

Guaranteeing Serializability by Two-Phase Locking

- If every transaction in a schedule follows the two-phase locking protocol, schedule guaranteed to be serializable
- Two-phase locking may limit the amount of concurrency that can occur in a schedule
- Some serializable schedules will be prohibited by two-phase locking protocol

Variations of Two-Phase Locking

- Basic 2PL
 - Technique described on previous slides
- Conservative (static) 2PL
 - Requires a transaction to lock all the items it accesses before the transaction begins
 - Predeclare read-set and write-set
 - Deadlock-free protocol
- Strict 2PL
 - Transaction does not release exclusive locks until after it commits or aborts

The **read-set** of a transaction is the set of all items that the transaction reads, and the **write-set** is the set of all items that it writes.

Variations of Two-Phase Locking (cont'd.)

- Rigorous 2PL
 - Transaction does not release any locks until after it commits or aborts
- Concurrency control subsystem responsible for generating read_lock and write_lock requests
- Locking generally considered to have high overhead

Concurrency Control Based on Timestamp Ordering

- Timestamp
 - Unique identifier assigned by the DBMS to identify a transaction
 - Assigned in the order submitted
 - Transaction start time
- Concurrency control techniques based on timestamps do not use locks
 - Deadlocks cannot occur

Concurrency Control Based on Timestamp Ordering (cont'd.)

- Generating timestamps
 - Counter incremented each time its value is assigned to a transaction
 - Current date/time value of the system clock
 - Ensure no two timestamps are generated during the same tick of the clock
- General approach
 - Enforce equivalent serial order on the transactions based on their timestamps

Concurrency Control Based on Timestamp Ordering (cont'd.)

- Timestamp ordering (TO)
 - Allows interleaving of transaction operations
 - Must ensure timestamp order is followed for each pair of conflicting operations
- Each database item assigned two timestamp values
 - read_TS(X)
 - write_TS(X)

Concurrency Control Based on Timestamp Ordering (cont'd.)

- Basic TO algorithm
 - If conflicting operations detected, later operation rejected by aborting transaction that issued it
 - Schedules produced guaranteed to be conflict serializable
 - Starvation may occur
- Strict TO algorithm
 - Ensures schedules are both strict and conflict serializable

Concurrency Control Based on Timestamp Ordering (cont'd.)

Strict Timestamp Ordering (TO):

- A variation of basic TO called strict TO ensures that the schedules are both strict (for easy recoverability) and (conflict) serializable.
- In this variation, a transaction T issues a $\text{read_item}(X)$ or $\text{write_item}(X)$ such that $\text{TS}(T) > \text{write_TS}(X)$ has its read or write operation delayed until the transaction T' that wrote the value of X (hence $\text{TS}(T') = \text{write_TS}(X)$) has committed or aborted.

Concurrency Control Based on Timestamp Ordering (cont'd.)

■ Thomas's write rule

- Modification of basic TO algorithm
- Does not enforce conflict serializability
- Rejects fewer write operations by modifying checks for write_item(X) operation
 1. If $\text{read_TS}(X) > \text{TS}(T)$, then abort and roll back T and reject the operation.
 2. If $\text{write_TS}(X) > \text{TS}(T)$, then do not execute the write operation but continue processing. This is because some transaction with timestamp greater than $\text{TS}(T)$ —and hence after T in the timestamp ordering—has already written the value of X . Thus, we must ignore the write_item(X) operation of T because it is already outdated and obsolete. Notice that any conflict arising from this situation would be detected by case (1).
 3. If neither the condition in part (1) nor the condition in part (2) occurs, then execute the write_item(X) operation of T and set $\text{write_TS}(X)$ to $\text{TS}(T)$.

THANK YOU